

Python for Software Development

Due: Friday, Sept. 25, 11:59 PM

Homework 1

This is an individual assignment.

Create a text file named `<your andrew id>_hw1.py` to store the answers to the problems below; for example, if you Andrew id is "fredw@andrew.cmu.edu", then name the file "fredw_hw1.py". Submit your solution on Gradescope.

Write a script that handles all four problems. Put a comment at the beginning with your name and Andrew id. Label each part of your code with a comment like this:

```
# Problem 1
```

The application you develop will keep track of university students' attendance at events, track their hours, and award points based on their participation. Test data (not all of it may be correct, see #1) will be stored in a data file named "students.csv", which is a CSV file of name, year, activity, time records. For each of the functions described below, use type hints for the parameters and return values.

1. Write a function named `parse_record( )` that accepts a single record string that contains a name, university year, name of an event, and hours attended. The function verifies/ changes values so that:

- no fields are empty
- the year is a valid integer
- the year is between 1 and 4
- the number of hours is a float
- the number of hours is greater than 0

If the record is valid (no empty fields), return the information as a dictionary with keys name, year, event, hours. Otherwise, return None, indicating the record is invalid.

2. Students earn participation points for events based on the number of hours they spend at an event.

Hours of Participation	Points Earned
1-2	10
3-5	20
6+	30

Write a function named `participation_points( )` that accepts the number of hours and returns the appropriate number of points.

3. The university classifies students based on their total participation hours. Write a function named `participation_standing( )` that accepts the hours a student worked, determines whether a student belongs to one of the following participation levels, and returns that str value (for example, return "Silver" for 7 hours).

Total Hours	Standing
1-5	Bronze
6-10	Silver
11-20	Gold
21+	Diamond

4. Create a function named `run_menu()` that accepts a list of student data and shows the following menu, which allows a user to:

- a. Search for a student by name.
- b. Display all students who participated in a specific event.
- c. Display all students in a specific participation standing.
- d. Quit the program.

The user should enter the letter associated with the choice. If the choice is incorrect, print “Invalid choice, please try again.” The menu should continue running until the user chooses to quit.

5. The university would like to recognize students who participated for at least 7 hours. Write a function named `filter_high_participation()` that takes a list of student data and returns a new list containing only students who participated for 10 or more hours. Solve this problem using a traditional loop. Then rewrite the solution using a list comprehension in a function named `filter_high_participation_comprehension()`. That is, you’ll have two functions that do exactly the same work but accomplish it in different ways.

6. Write a function named `get_names_loop()` that takes the list of student data and create and returns a list containing only student names. Solve the problem using a traditional loop. Then rewrite the solution to the same problem using `map()` and a lambda function; name this function `get_names_map()`.

Repeat the process to create a list containing only participation hours – that is, two functions named `get_hours_loop()` and `get_hours_map()`.

7. The university wants to know which events were the most popular. Write functions to compute the following statistics:

- how many students participated in each event; name the function `event_statistics()`, which takes the student data as a parameter and returns a map of {event name:number of students} for each event.
- the most popular event; name the function `most_popular_event()`, which takes the map described above as the parameter and returns the str name.
- the least popular event; name the function `least_popular_event()`, similar to the above.

Your solution should not hardcode event names. It should work correctly even if new events are added later.

8. Create a function named `generate_report()`, which takes the student data as a parameter; it prints a formatted report for every activity record.

A sample report entry should look similar to:

```
Student: Alice
Year: 1
Event: Hackathon
Hours: 6
Points: 30
Standing: Silver
```

The function should support each valid entry.

9. Write a main program that uses your functions to do the following:

- read the data file into a list of str values
- parses each record to create the student data list of dict's
- gets and prints the statistics, most popular event, and least popular event
- generate the report
- run the interactive menu

Documentation: Every Python file must have your name and Andrew ID at the top as a comment. Every function must have a one-line (minimum) statement of purpose.

Deliverables: Submit only your .py file named `<your andrew id>_hw1.py` on Gradescope.